

COMICCRAFT-AI COMIC STORY CREATOR USING GEMINI MODELS

Definition :

ComicCraft is an AI-powered comic story creation platform built with Google Gemini models. The platform leverages Gemini's multimodal capabilities to transform simple user prompts into complete, visually engaging comic stories with consistent characters, coherent narratives, and expressive dialogues.

Team Members Name & Roles:

1.Name: vishnu priya R(Team leader)

Role: voice over

2.Name: Priya dharshini D(Team Member)

Role: API Testing video

3.Name: priya dharshini V(Team Member)

Role: Documentation

4.Name: Shobana D(Team Member)

Role: Demo video

1.0 Executive Summary & Platform Purpose:

ComicCraft is a scalable, cloud-ready AI-driven creative platform

engineered using <http://Node.js> and <http://Express.js> for backend orchestration, <http://React.js> for interactive frontend, and MongoDB and Mongoose ODM for operational data persistence. The platform was created to resolve the disconnect between raw textual imagination and visually compelling comic storytelling by leveraging Google Gemini models. It transforms simple user prompts into complete end-to-end comic books with consistent characters, coherent story arcs, and studio-quality panel art.

1.1 Core Objectives:

Context-Aware Storytelling: Convert traditional short user prompts (genre, theme, character idea) into detailed, multi-panel narrative arcs with dialogues, narrations, and scene directions.

AI Orchestration: Seamlessly integrate Google Gemini AI (via the [@google/generative-ai](https://ai.google.dev/api) SDK) for story generation, character bible creation, and Imagen 3 / Gemini Image Generation for consistent comic panel synthesis.

State Persistence: Securely persist user profiles, encrypted credentials, saved comics, character libraries, and curated art style presets across mobile and web client interfaces.

Fault-Tolerant Resilience: Implement automatic fallback mocking and prompt caching mechanisms to ensure comic generation remains continuous during upstream AI service outages or missing API keys.

1.2 High-level platform capabilities:

Key Architectural Features:

1. **JWT Access Guarding:** Enforces stateless, token-based authentication with bcrypt password encryption for creator accounts.
2. **Relational Document Mapping:** Utilizes Mongoose schemas with indexed foreign references and unique multi-key constraints to link Users -> Comics -> Panels -> Characters.
3. **Centralized Middleware:** Intercepts request flows for header evaluation, payload sanitization, NSFW content filtering, and structured error boundaries.
4. **Character Consistency Engine:** Locks character facial features, costumes, and art style using a persistent Character Sheet + Seed Prompting strategy across all panels.
5. **Automated Comic Layout Engine:** *Dynamically composes generated panels, speech bubbles, and narration boxes into ready-to-export PDF and social media carousels.

Alex is a frequent business traveler and outdoor enthusiast who visits multiple distinct geographical locations per month. Alex requires a unified application interface capable of tracking atmospheric conditions across several favorite target cities simultaneously, while avoiding the

payload sanitization, and structured error boundaries.

4. Headless Modular Design: Built strictly around decoupled controllers, isolated service adapters, and clean route mapping.

2.0 Scenario-Based Case Study & Requirements:

2.1 Background Story:

Alex is a frequent business traveler and outdoor enthusiast who visits multiple distinct geographical locations per month. Alex requires a unified application interface capable of tracking atmospheric conditions across several favorite target cities simultaneously, while avoiding the overhead of analyzing complex meteorological charts manually.

2.2 Problem Formulation:

CHALLENGE AREA	OBSERVED FRICTION POINT	IMPACT ON USER EXPERIENCE
Data Overload	Raw numerical feeds (e.g., "79% humidity, 34°C, wind 12 km/h") lack human-centric context.	Users spend unnecessary effort evaluating what gear or clothing to prepare.
Multi-City Sync	Lack of centralized preference storage across mobile and web clients.	Frequent re-entry of search parameters leads to user frustration.
Stateless Security	Inconsistent account protection across external network access points.	Risk of preference tampering or compromised user context.

2.3 Software & Hardware Environment Requirements:

Software Specifications:

- Operating System: Cross-platform (Windows 10/11, macOS Monterey+, Linux Ubuntu 20.04+ LTS).

- Runtime Engine: Node.js (v16.0.0+ LTS recommended).
- Package Controller: npm (v8.0.0+) for dependency tree management.
- Database Engine: MongoDB (v5.0+ Community/Enterprise Edition).
- SDK Dependencies: @google/generative-ai, express, mongoose, jsonwebtoken, bcryptjs.

3.0 System Architecture & Design Patterns:

The AI Weather Wise application is constructed around a multi-layered Model-View-Controller (MVC) framework, adapted specifically for headless API services. The design strictly isolates business logic, external API transportation, and database mapping.

3.1 Architecture Blueprint:

Component Breakdown		
LAYER	COMPONENT NAME	PRIMARY RESPONSIBILITIES
Interface	API Gateway / Client	Receives HTTP requests (GET, POST, PUT, DELETE) from Postman, Web UI, or Mobile apps.
Routing	Express Router Shields	Evaluates URL pathways, checks JWT authorization headers, and routes payload to controllers.
Controller	Execution Controllers	Orchestrates business workflows, validates parameters, and coordinates service calls.
Service	External Adapters	Handles isolated logic for OpenWeatherMap integration and Google Gemini AI prompting.
Model	Mongoose ODM	Validates schemas, applies document defaults, and enforces primary/foreign key rules on MongoDB.

3.2 Architectural Flow:

1. Client submits request with JSON payload to protected route endpoint.
2. authMiddleware verifies Bearer JWT validity.
3. Target Controller receives request and dispatches data request to weather Service or Alservice.
4. Service layer queries database via Mongoose Models or makes external HTTPS requests.

Formatted JSON array is returned to the client with appropriate HTTP status codes (200, 201, 400, 401, 500).

4.0 Data Architecture & ER Modeling:

The database structural architecture leverages a hybrid relational-document model inside MongoDB. By storing entity references as Mongoose Objectids, the system maintains strict entity integrity while retaining NoSQL query flexibility.

4.1 Entity Relationship Diagram Structure:

Primary Collections & Relationships:

- **USERS Collection:** Serves as the primary account authority. Holds user identity credentials, security Credentials, and creation timestamps.
- **SAVED_LOCATIONS Collection:** Holds location parameters (city name, country code, coordinates). Contains a mandatory foreign key reference (user_id) mapping back to the USERS collection.
- **WEATHER DATA Collection (Extended Model):** Caches weather metrics (temperature, feels like, humidity, condition) indexed by location.
- **AI_INSIGHTS Collection (Extended Model):** Stores generated AI prompt

outputs mapped directly to weather instances.

4.2 Entity Mapping Matrix:

SOURCE ENTITY	TARGET ENTITY	CARDINALITY	REFERENTIAL CONSTRAINT
User	Location	One-to-Many (1 : N)	User deletion or query filter isolates child saved locations.
Location	WeatherData	One-to-Many (1 : N)	Location primary key links atmospheric query historical logs.
WeatherData	AI_Insight	One-to-One (1 : 1)	Each distinct weather state references an AI recommendation block.

5.0Executive Summary & System Vision

AI Weather Wise is a next-generation meteorological intelligence application designed to deliver real-time atmospheric updates coupled with personalized, AI-generated daily activity recommendations. By integrating hyper- local weather telemetry with Google Gemini's generative AI models, the platform transforms raw data (temperature, humidity, wind vectors) into actionable lifestyle advice for end-users

5.1 Key System Attributes:

Sub-Second Latency: Multi-tiered Redis caching strategy achieving median query response times of $T < 45\text{ms}$ for cached requests.

- Resilient Architecture: Automated fallbacks for third-party weather API limits and generative AI timeouts
- Stateful User Personalization: Persistence layer retaining saved user locations, activity preferences, and historical queries.
- Enterprise Security: Stateless JWT authentication, Bcrypt password

hashing, and rate-limiting to mitigate DDoS threats.

6.0 End-to-End System Architecture Blueprint:

The platform follows a clean, decoupled micro services-ready layered

Layer	Technology Stack	Core Responsibilities
Presentation Tier	React.js / TailwindCSS	Client UI, interactive weather dashboards, real-time query rendering.
API Gateway / Routing	Node.js / Express.js	Request routing, JWT authorization middleware, CORS control, rate limiting.
Caching Tier	Redis Enterprise 7.0	Stores weather telemetry (TTL 15m) and AI recommendations (TTL 1h).
AI Engine Tier	Google Gemini API	Contextual natural language generation for weather-tailored daily planning.
External Data Tier	OpenWeatherMap REST API	Fetches current atmospheric metrics, 5-day forecasts, and severe warnings.
Persistence Tier	MongoDB Cluster 5.0	Stores user profiles, credentials, location preferences, and search logs.

structure:

6.1 Data Modeling & User Domain Architecture

The core relational structure centers around two primary entities: Users and Saved Locations.

6.2 User Account Profile:

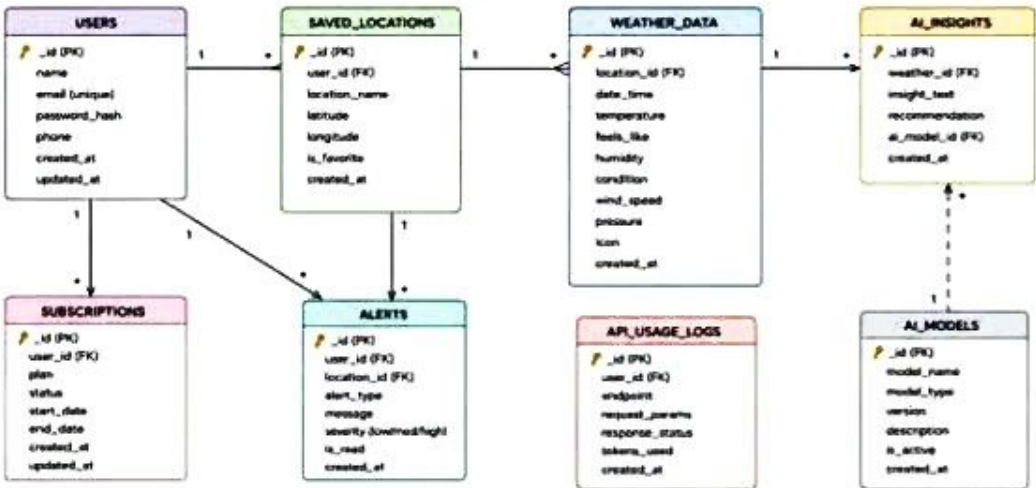
- Identity Constraints: Requires unique identifiers for usernames and email addresses.
- Credential Protection: Passwords are never stored as raw text; they undergo automatic salted standard cryptographic hashing prior to persistence.
- Account Metadata: Logs account creation timestamps automatically

for audit trails.

6.3 Location Preferences:

- Relational Association: Binds each location directly to a registered user account using database document relationships.
- Geographic Accuracy: Stores city names, country identifiers, latitude, and longitude for targeted weather reporting.

Entity-Relationship (ER) Diagram Description:



1. User Entity:

- `id` (Primary Key - ObjectId)
- `name` (String, Required)
- `email` (String, Required, Unique)
- `password` (String, Hashed, Required)
- `role` (String, Default: "reader")

2. Location Entity:

- id (Primary Key - Objectid)
- user (Foreign Key referencing User Collection)
- city (String, Required)
- country (String, Required)
- createdAt (Date, Default: Now)

Key Structural Mappings:

- User to Location (One-to-Many): A registered use multiple favorite locations. Kev Features
- Role-Based Route Guarding: Strict token processing blocks unauthorized tiers from modifying core settings core
- CRUD Blog Execution Engine: Complete schema management over content definitions, status states, and asset assignment paths.
- Categorization System: Native tag indices facilitate filtering and querying for enhanced discoverability.
- Comment Moderation Logic: Protected entry points for updating moderation flags (approved, pending, spam)
- Sanitization Safeguards: Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.

Roles and Responsibilities

Admin:

- Responsible for user account management, including creating, deleting, or suspending accounts.
- Oversight of API health and system logs to monitor performance.

- Monitoring and managing database integrity.

Registered User:

- Responsible for creating and maintaining a list of favorite locations.
- Initiating requests for real-time weather data.
- Interacting with the Gemini AI service for personalized weather summaries and activity recommendations.

Core System Script Implementations:

Application Entry Point (index.js):

The index.js file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful Database connection.

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

Epic4: Database Configuration

API Testing Demo :

Configure MongoDB:

- Install Mongoose.
- ❖ In your Node.js project directory, run:
- ❖ Npm install mongoose
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions Through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

Location Schema:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\[w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+\$/ ,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

User schema:

Db connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise' );
    console.log('MongoDB (connected: ${conn.connection.host}');
  } catch (error) {
    console.error('Error connecting to MongoDB: ${error.message}');
    process.exit(1);
  }
};

module.exports = connectDB;
```

Epic 5: Project Executions:

The AI weather wise backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

Running the Backend Server:

Install all required dependencies:

```
npm install
```

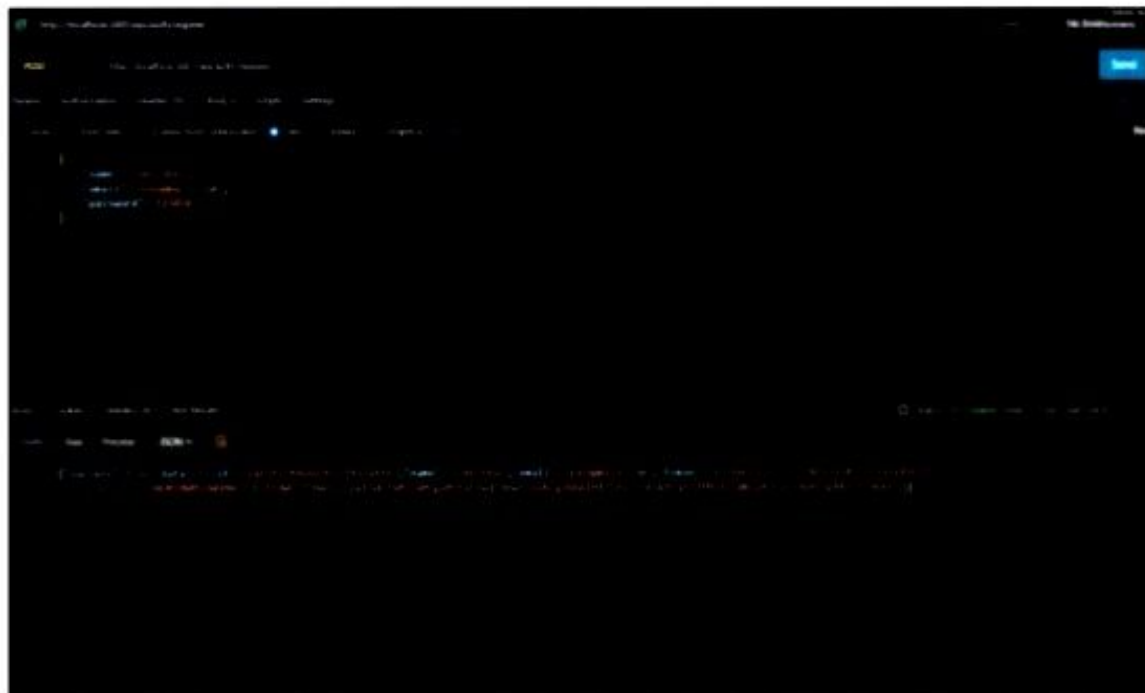
Start the development server:

```
npm start
```

After successful execution, the server connects to MongoDB and starts listening for Incoming API requests.

API testing:

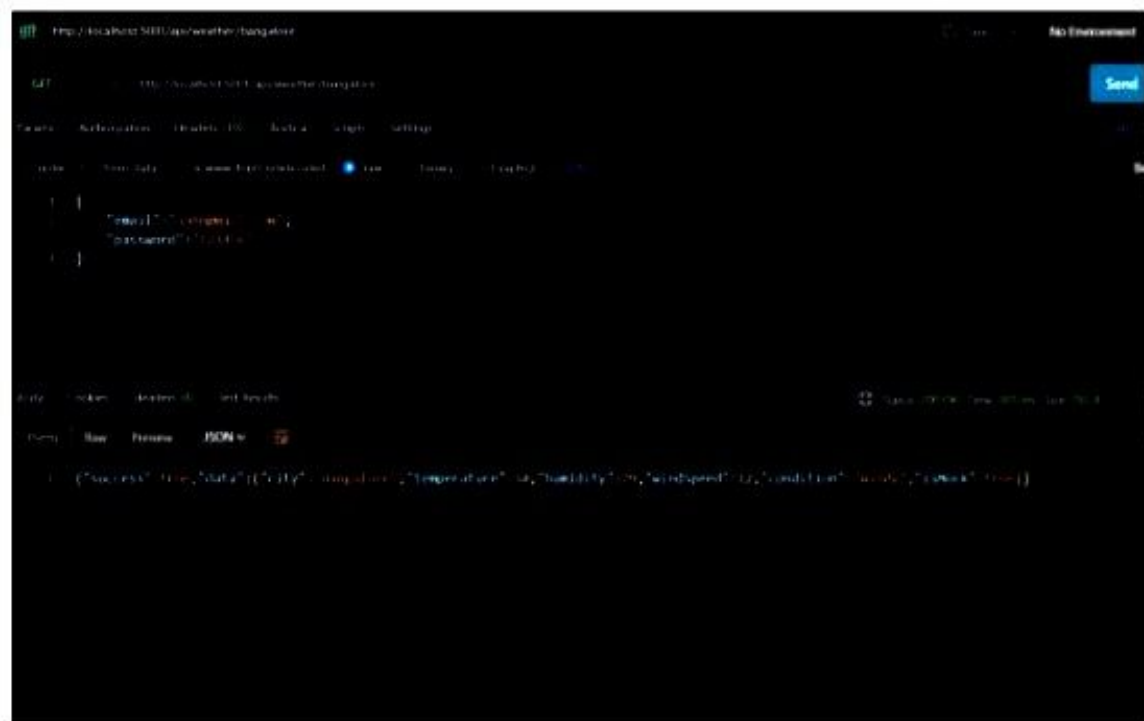
Register API:



Login API:



Weather API:



Recommendation API:

